THEORY

INTRODUCTION TO MACHINE LEARNING

1. What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and use those patterns to make predictions or decisions without requiring programmers to write a separate rule for every possible situation.

Real-Life Example: House Price Prediction

Imagine a real estate company wants to predict house prices using historical data.

Area (sq. ft.)	Bedrooms	House Price
800	2	₹25 lakh
1000	2	₹32 lakh
1200	3	₹40 lakh
1500	3	₹50 lakh
1800	4	₹62 lakh

The ML algorithm learns patterns from these examples. After training, the model can estimate the price of a new house.

The prediction is an estimate, not a guaranteed market price.

Important Terms

- Data: Information used to train or evaluate a model.
- Feature: An input variable used for prediction, such as house area.
- Target or label: The value the model must predict, such as house price.
- Algorithm: A method used to learn patterns from data.
- Model: The learned result of training an algorithm.

- Prediction: The output generated for new input data.

Traditional Programming vs Machine Learning

Traditional programming:

- Programmer writes rules.
- Input data is processed using those rules.
- The program generates an output.

Machine Learning:

- Provide examples and a learning algorithm.
- The algorithm learns patterns from training data.
- The trained model predicts outcomes for new data.

2. Machine Learning Workflow

A typical Machine Learning workflow consists of these steps:

1. Data collection: Collect relevant records.
2. Data preparation: Clean missing values, duplicates and incorrect entries.
3. Dataset splitting: Separate training and testing data.
4. Model training: Allow the algorithm to learn patterns.
5. Model evaluation: Measure performance on unseen data.
6. Prediction: Use the trained model on new data.
7. Monitoring: Check performance as new data becomes available.

Real-Life Example: Student Marks Prediction

A coaching centre collects students' study hours and examination marks.

- Collect historical student records.
- Clean the dataset.
- Use some records for training.

- Use separate records for testing.
- Evaluate prediction errors.
- Predict marks for a new student.

Study hours alone cannot reliably predict examination results because prior knowledge, preparation quality, sleep and exam difficulty also matter.

3. Types of Machine Learning

A. Supervised Learning

Supervised Learning uses labelled data. Each training example includes input features and the correct target.

Two common tasks are:

Regression

Regression predicts a numerical value.

Examples:

- Predict house prices.
- Forecast sales.
- Estimate delivery time.
- Predict electricity consumption.

Classification

Classification predicts a category or class.

Examples:

- Spam or Not Spam.
- Fraudulent or Legitimate transaction.
- Iris flower species.
- Positive or Negative customer review.

B. Unsupervised Learning

Unsupervised Learning uses data without supplied target labels. It attempts to discover patterns or structure.

Real-life example: Customer segmentation.

An online shop groups customers based on spending, order frequency and purchase behaviour.

Possible groups:

- Frequent high-spending customers.
- Occasional buyers.
- Customers who mainly purchase during discounts.

The algorithm discovers groups; people must interpret whether those groups are useful.

Supervised vs Unsupervised Learning

Feature	Supervised	Unsupervised
Target labels	Available	Not supplied
Purpose	Predict a target	Discover patterns
Tasks	Regression, classification	Clustering, dimensionality reduction
Example	House price prediction	Customer segmentation
Algorithms	Linear Regression, Naive Bayes	K-Means, PCA

Reinforcement Learning is another major ML approach, but it is outside the scope of these introductory categories.

4. Dataset Splitting

Dataset splitting separates data used to train a model from data used to evaluate it.

Why is splitting important?

A model might memorize training examples without learning patterns that generalize to new data. Testing on unseen records helps assess generalization.

Common Splitting Ratios

Training	Testing	Usage
80%	20%	Common starting point
70%	30%	Larger test set
90%	10%	Larger training set when the test set remains adequate

The appropriate ratio depends on dataset size and the problem.

Python Example

```
import pandas as pd
from sklearn.model_selection import train_test_split

data = {
    "study_hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "marks": [20, 25, 35, 40, 48, 55, 65, 72, 82, 90]
}

df = pd.DataFrame(data)

X = df[["study_hours"]]
y = df["marks"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

Explanation:

- X contains input features.
- y contains the target.
- test_size=0.2 reserves 20% for testing.
- random_state=42 makes the split reproducible.
- X_train and y_train are used for training.
- X_test and y_test are used for evaluation.

For time-series data such as monthly sales, a chronological split is generally more appropriate than a random split.

5. Model Evaluation

Model evaluation measures how well a trained model performs on unseen data.

A. Regression Metrics

Regression predicts numerical values.

Mean Absolute Error (MAE)

MAE calculates the average absolute difference between actual and predicted values.

Example:

Actual values: 20, 30, 40

Predicted values: 22, 28, 43

Absolute errors: 2, 2, 3

MAE = (2 + 2 + 3) / 3 = 2.33

Lower MAE generally indicates better predictions when comparing models on the same target and evaluation dataset.

Mean Squared Error (MSE)

MSE squares prediction errors before calculating their average. Large errors receive greater penalties.

R^2 Score

R^2 measures how well the model explains target variation relative to predicting the target mean.

- 1: Perfect predictions on the evaluated data.
- 0: Equivalent to the mean-prediction baseline under the standard definition.
- Negative: Worse than that baseline.

B. Classification Metrics

Classification predicts categories.

Consider a spam detector:

Actual Class	Predicted Spam	Predicted Not Spam
Spam	80	20
Not Spam	10	90

- True Positive (TP): 80 spam emails correctly identified.
- False Negative (FN): 20 spam emails missed.
- False Positive (FP): 10 legitimate emails incorrectly flagged.
- True Negative (TN): 90 legitimate emails correctly identified.

Accuracy

Accuracy = (TP + TN) / Total predictions

Accuracy = (80 + 90) / 200 = 85%

Precision

Precision = TP / (TP + FP)

Precision = 80 / 90 = 88.89%

Recall

Recall = TP / (TP + FN)

Recall = 80 / 100 = 80%

F1-score

F1-score is the harmonic mean of precision and recall.

Choose evaluation metrics based on the business problem. Accuracy alone may be misleading when classes are imbalanced.

6. Practical 1: House Price Prediction Using Linear Regression

What is Linear Regression?

Linear Regression is a supervised learning algorithm used to predict numerical values by fitting a linear relationship between features and a target.

For a single input feature:

$\hat{y} = b_0 + b_1 x$

Where:

- $\hat{y}$: Predicted value.
- x: Input feature, such as house area.
- b_0: Intercept.
- b_1: Learned coefficient or slope.

Python Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

data = {
```

```
    "area": [500, 600, 700, 800, 900, 1000, 1100, 1200],
    "price": [15, 18, 21, 24, 27, 30, 33, 36]
}

df = pd.DataFrame(data)

X = df[["area"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Actual prices:", y_test.tolist())
print("Predicted prices:", predictions.round(2).tolist())

print("MAE:", mean_absolute_error(y_test, predictions))
print("R2:", r2_score(y_test, predictions))

new_house = pd.DataFrame({"area": [850]})
print("Predicted price:", model.predict(new_house)[0], "lak
h")
```

Explanation:

- LinearRegression() creates the model.
- fit() trains it.
- predict() generates predictions.
- MAE measures average absolute error.
- R^2 measures model performance relative to the mean baseline.

This is a deliberately simple dataset. Real house prices depend on location, condition, age and other factors.

7. Practical 2: Iris Flower Classification

What is the Iris Dataset?

The Iris dataset is a classic classification dataset containing four flower measurements:

- Sepal length
- Sepal width
- Petal length
- Petal width

The model predicts one of three species: Setosa, Versicolor or Virginica.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_re
port

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))

print(classification_report(
    y_test,
    predictions,
    target_names=iris.target_names
))

new_flower = [[5.1, 3.5, 1.4, 0.2]]
predicted_class = model.predict(new_flower)[0]

print("Predicted species:", iris.target_names[predicted_clas
s])
```

Key learning points:

- load_iris() loads the dataset.
- DecisionTreeClassifier() creates the classifier.
- stratify=y helps preserve class proportions.
- accuracy_score() calculates accuracy.
- classification_report() displays precision, recall and F1-score.

The test score applies to this particular split, not every future dataset.

8. Practical 3: Spam Detection Using Naive Bayes

What is Naive Bayes?

Naive Bayes is a family of supervised classification algorithms based on Bayes' theorem. It uses a simplifying conditional-independence assumption.

Multinomial Naive Bayes is commonly used for word-count features in text classification.

Python Code

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report

emails = [
    "Win a free prize now",
    "Claim your free gift today",
    "Congratulations you won a reward",
    "Limited offer buy now",
    "Meeting scheduled for tomorrow",
    "Please send the project report",
    "Let us discuss the assignment",
    "Your class starts at ten",
    "Can we reschedule the meeting?",
    "Please review the attached document"
]

labels = [
    "spam", "spam", "spam", "spam",
    "not_spam", "not_spam", "not_spam",
    "not_spam", "not_spam", "not_spam"
]

X_train, X_test, y_train, y_test = train_test_split(
    emails, labels, test_size=0.3,
    random_state=42, stratify=labels
)

model = make_pipeline(
    CountVectorizer(),
    MultinomialNB()
)
```

```
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(classification_report(
    y_test, predictions, zero_division=0
))

new_emails = [
    "Claim your free reward now",
    "Please attend the project meeting"
]

print(model.predict(new_emails))
```

How it works:

1. CountVectorizer converts text into word-count features.
2. MultinomialNB learns patterns associated with each class.
3. make_pipeline combines text processing and classification.
4. fit() trains the pipeline.
5. predict() classifies new emails.

The dataset is intentionally tiny and only suitable for teaching. A production spam detector needs more representative data and rigorous testing.

WORKING WITH REAL DATA

1. Data Cleaning and Preprocessing

Data cleaning identifies and corrects problems in datasets.

Data preprocessing is the broader process of transforming raw data into a suitable format for analysis and Machine Learning.

Real-Life Example: Student Dataset

Student	Study Hours	Attendance	Marks
Rahul	4	90	78
Priya	3	85	72
Amit	Missing	80	65
Priya	3	85	72
Riya	5	110	88

Problems:

- Amit's study hours are missing.
- Priya's record is duplicated.
- Riya's attendance may be invalid because it exceeds 100%.

Investigate these issues before training a model.

Common Data-Quality Problems

Problem	Example	Possible Treatment
Missing values	Blank study hours	Impute or investigate
Duplicates	Repeated student record	Verify and remove genuine duplicates
Inconsistent text	Kolkata, kolkata	Standardize if equivalent
Incorrect type	Price stored as text	Convert after checking format
Invalid value	Negative age	Investigate source
Outlier	Extremely high sales	Investigate; do not assume it is an error

2. Handling Missing Values

Missing values indicate that a field has no recorded value.

Common approaches:

1. Remove rows when justified.
2. Use the mean for suitable numerical data.
3. Use the median for skewed numerical data or data with outliers.

4. Use the mode for suitable categorical data.
5. Use a missing category or advanced imputation method when appropriate.

Do not automatically replace every missing value with zero.

Python Example

```python
import pandas as pd

data = {
    "name": ["Rahul", "Priya", "Amit", "Riya"],
    "study_hours": [4, 3, None, 5],
    "marks": [78, 72, 65, None]
}

df = pd.DataFrame(data)

print("Missing values before cleaning:")
print(df.isnull().sum())

df["study_hours"] = df["study_hours"].fillna(
    df["study_hours"].median()
)

df["marks"] = df["marks"].fillna(
    df["marks"].median()
)

print("\nCleaned dataset:")
print(df)

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

Explanation:

- isnull() identifies missing values.
- sum() counts missing values.
- median() calculates the median.
- fillna() replaces missing values.

In ML workflows, calculate imputation values from training data only and apply the learned values to test data.

3. Removing Duplicate Records

Duplicate records can distort statistics and model training. Verify whether records represent genuine duplicates before removing them.

Python Example

```
import pandas as pd

data = {
    "student_id": [101, 102, 102, 103],
    "name": ["Rahul", "Priya", "Priya", "Amit"],
    "marks": [78, 85, 85, 65]
}

df = pd.DataFrame(data)

print("Duplicate rows:")
print(df[df.duplicated()])

df_cleaned = df.drop_duplicates()

print("\nAfter removing duplicates:")
print(df_cleaned)
```

- duplicated() identifies repeated rows.
- drop_duplicates() removes subsequent duplicate rows by default.

If student IDs should be unique, investigate duplicates based on student_id instead of assuming all repeated rows are invalid.

4. Feature Selection

Feature selection means choosing useful input variables for a Machine Learning model.

Example: Student Marks Prediction

Potential features:

- Study hours
- Attendance
- Previous examination marks

Possible non-useful features:

- Student name
- Random student ID

Target:

- Final examination marks

Never use the target itself as an input when predicting that same target. This causes data leakage.

Python Example

```
import pandas as pd

df = pd.DataFrame({
    "student_id": [101, 102, 103, 104],
    "study_hours": [2, 4, 5, 7],
    "attendance": [70, 80, 90, 95],
    "previous_marks": [50, 60, 70, 80],
    "final_marks": [55, 65, 75, 88]
})
```

```
X = df[[
    "study_hours",
    "attendance",
    "previous_marks"
]]

y = df["final_marks"]

print("Input features:")
print(X)

print("\nTarget:")
print(y)
```

Advanced feature selection methods include correlation analysis, permutation importance and feature selection tools available in scikit-learn.

Correlation does not prove causation. Evaluate feature choices using domain knowledge and unseen data.

5. Feature Scaling

Feature scaling transforms numerical variables to comparable scales.

Example features:

- Age: 25
- Monthly income: 50000
- Credit score: 720

Some algorithms are sensitive to feature scales.

A. Standardization

z = (x - mean) / standard deviation

StandardScaler transforms a feature using its training-set mean and standard deviation.

B. Min-Max Scaling

x_scaled = (x - minimum) / (maximum - minimum)

MinMaxScaler typically maps values to a range between 0 and 1.

When is Scaling Useful?

Algorithm	Scaling Usually Needed?
Linear Regression	Not always
Logistic Regression	Often useful
K-Nearest Neighbours	Usually important
Support Vector Machine	Usually important
K-Means	Usually important
Decision Tree	Generally not required
Random Forest	Generally not required

Python Example

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScale
r

df = pd.DataFrame({
    "age": [20, 30, 40, 50],
    "income": [20000, 40000, 60000, 80000]
})

standard_scaler = StandardScaler()
standardized = standard_scaler.fit_transform(df)

minmax_scaler = MinMaxScaler()
normalized = minmax_scaler.fit_transform(df)

print("Original data:")
print(df)
```

```
print("\nStandardized data:")
print(standardized.round(2))

print("\nMin-max scaled data:")
print(normalized.round(2))
```

Important: Fit scalers using training data only, then use transform() on testing data. This prevents information leakage.

6. Introduction to Scikit-learn Pipelines

A Machine Learning pipeline connects preprocessing steps and a model into one workflow.

Example:

1. Fill missing numerical values.
2. Scale numerical features.
3. Train a classification model.
4. Apply the same preprocessing to new data.

Pipelines reduce inconsistencies between training and prediction.

Python Example: Student Pass/Fail Prediction

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_re
port

df = pd.DataFrame({
```

```
    "study_hours": [
        1, 2, 2, 3, 4, 4, 5, 6, 7, 8,
        1, 3, 5, 6, 7, 8, 2, 4, 6, 9
    ],
    "attendance": [
        40, 50, 55, 60, 65, 70, 75, 80, 85, 90,
        45, 62, 72, 78, 82, 95, 52, 68, 88, 96
    ],
    "result": [
        "Fail", "Fail", "Fail", "Fail", "Pass",
        "Pass", "Pass", "Pass", "Pass", "Pass",
        "Fail", "Fail", "Pass", "Pass", "Pass",
        "Pass", "Fail", "Pass", "Pass", "Pass"
    ]
})

X = df[["study_hours", "attendance"]]
y = df["result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25,
    random_state=42, stratify=y
)

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

pipeline.fit(X_train, y_train)

predictions = pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
```

```
print(classification_report(
    y_test, predictions, zero_division=0
))

new_student = pd.DataFrame({
    "study_hours": [5],
    "attendance": [80]
})

print("Predicted result:", pipeline.predict(new_student)[0])
```

Explanation:

- SimpleImputer fills missing numerical values.
- StandardScaler standardizes features.
- LogisticRegression performs classification.
- Pipeline connects the steps.
- pipeline.predict() applies preprocessing before generating a prediction.

The sample dataset is synthetic and too small to validate a real student-outcome prediction system.

7. Practical Project: COVID-19 Dataset Analysis

Objective

Learn to load, clean, summarize and visualize a public-health dataset.

Use a documented public dataset with clear definitions, reporting dates and source information.

Suggested Tasks

1. Load a CSV file.
2. Inspect column names and data types.
3. Identify missing values and duplicates.

4. Convert dates to a consistent format.
5. Summarize reported cases by date or region.
6. Plot trends using Matplotlib.
7. Document the limitations of the dataset.

Python Starter Code

Assume the CSV file contains date, country and new_cases columns.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("covid_data.csv")

print(df.head())
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

df["date"] = pd.to_datetime(df["date"], errors="coerce")
df["new_cases"] = pd.to_numeric(
    df["new_cases"], errors="coerce"
)

df = df.dropna(subset=["date", "new_cases"])
df = df.drop_duplicates()

country_data = df[df["country"] == "India"].copy()
country_data = country_data.sort_values("date")

plt.figure(figsize=(10, 5))
plt.plot(country_data["date"], country_data["new_cases"])
```

```
plt.xlabel("Date")
plt.ylabel("Reported new cases")
plt.title("Reported COVID-19 Cases in India")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

This is a starter analysis, not a complete epidemiological study. Reporting revisions, missing dates and changes in reporting practices may require additional investigation.

8. Practical Project: Sales Prediction Using CSV Data

Objective

Train a regression model using historical sales data.

The example assumes that sales may be associated with advertising expenditure. The sample records below are synthetic.

Step 1: Create sales_data.csv

Save the following content as sales_data.csv.

```
advertising_spend,sales
1000,5000
1500,6200
2000,7100
2500,8500
3000,9600
3500,10800
4000,11900
4500,13100
5000,14000
5500,15400
6000,16500
6500,17800
```

Step 2: Train and Evaluate the Model

```python
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

df = pd.read_csv("sales_data.csv")

print("Dataset preview:")
print(df.head())

print("\nMissing values:")
print(df.isnull().sum())

X = df[["advertising_spend"]]
y = df["sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, predictions))
print("R2 score:", r2_score(y_test, predictions))

new_budget = pd.DataFrame({
    "advertising_spend": [3200]
})

predicted_sales = model.predict(new_budget)
```

```
print("Predicted sales:", predicted_sales[0])
```

Step 3: Discuss Business Limitations

Sales may also depend on:

- Product pricing.
- Discounts and promotions.
- Seasonality.
- Competitor activity.
- Stock availability.
- Customer demand.

The model identifies patterns in historical data; it does not prove that increasing advertising expenditure will cause sales to increase.

For genuine sales forecasting, preserve chronological order when splitting the dataset and consider other relevant features.